

Into the Tidyverse

Today's agenda:

- Tidyverse

tidyverse

- A collection of R packages built on a similar syntax
- Functions/packages designed to work together
- Very useful for data wrangling and data visualization
- Many people find them more intuitive than base R

Tidyverse update!

<https://hadley.github.io/genzplyr/index.html>

Genzplyr

- Does what dplyr does, but using Gen Z slang
- Use on Assignment 4 for 1% extra credit



dplyr

- Used for data wrangling
- More readable than base R (to most)
- Downside is that the syntax is still evolving
- Designed for use with pipes: %>% or |>

dplyr

dplyr functions:

- Take in a data.frame (or tibble)
- Do something
- Return a data.frame (or tibble)

This means that you can easily chain functions together with pipes!

```
data |>  
do_something() |>  
do_something_else()
```

Getting data

We'll work with the Avonet data again

```
avo <- read_rds("https://tinyurl.com/avonetdata")
```

Go ahead and load the data now

Operating on rows

We're going to start by talking about function that operate on rows:

`filter()`

`arrange()`

`distinct()`

Filtering data

base R:

```
parrot_fam1 <- avo[which(avo$Family1 == "Psittacidae"),]
```

dplyr:

```
parrot_fam2 <- avo |>  
  filter(Family1 == "Psittacidae")
```

Try these now and see if you notice any differences

Filtering on multiple columns

In R we can use

- “&” (AND) to specify that multiple conditions have to be met

- “|” (OR) (vertical line) to specify that either condition can be met

This can be used to make our filtering more complex

(Note that my favorite Star Wars media is thus “&|”)

Filtering on multiple columns: &

```
forest_parrots1 <- avo[which(avo$Family1 == "Psittacidae" &  
                             avo$Habitat == "Forest"),]
```

```
forest_parrots2 <- avo |>  
  filter(Family1 == "Psittacidae" &  
         Habitat == "Forest")
```

```
forest_parrots3 <- avo |>  
  filter(Family1 == "Psittacidae") |>  
  filter(Habitat == "Forest")
```

Are these all equivalent? Try them and find out

Filtering on multiple columns: &

Your turn!

Filter the avo dataset to only include:

- A single order (of your choice)

- A single trophic level

Filtering on multiple columns: |

```
parrots_or_cockatoos <- avo %>%  
  filter(Family1 == "Psittacidae" |  
         Family1 == "Cacatuidae")
```

Alternative using “%in%” and a vector:

```
parrots_or_cockatoos2 <- avo %>%  
  filter(Family1 %in% c("Psittacidae", "Cacatuidae")  
         )
```

Try running these now

Filtering on multiple columns: & and |

Let's say we wanted to only keep forest parrots OR woodland cockatoos

- We need to make our filter more complex
- We can combine &, |, and () to achieve this:

```
fp_or_wc <- avo |>
  filter((Family1 == "Psittacidae" &
          Habitat == "Forest")|
         (Family1 == "Cacatuidae" &
          Habitat == "Woodland"))
```

Try this now! What would happen if we forgot the grouping parentheses?

Filtering on multiple columns: & and |

Your turn:

Make a version of the data that only contains:

Forest Carnivores in either the family "Accipitridae" or "Falconidae"

Sorting your data

We can use “arrange” to sort by a column

```
avo_by_mass <- avo %>%  
  arrange(Mass)
```

```
avo_by_mass_dec <- avo %>%  
  arrange(-Mass)
```

```
avo_by_mass_dec2 <- avo %>%  
  arrange(desc(Mass))
```

Use this syntax to figure out which bird has the longest wings

distinct

Used to remove duplicate rows

```
avo_no_dupes <- avo %>%  
  distinct()
```

Can also be used on particular fields, combinations of fields

```
avo_fams <- avo %>%  
  distinct(Family1)
```

```
family_trophic_levels <- avo %>%  
  distinct(Family1, Trophic.Level)
```

What do these two data frames contain?

Operating on columns

There are also function that operate on columns:

`mutate()`

`rename()`

`select()`

`relocate()`

Mutate

- Can be used to create new variables or modify existing ones
- Let's create a variable calculating the difference bt beak length measures:

```
avo_w_beak_diff <- avo %>%  
  mutate(beak_length_difference =  
    Beak.Length_Culmen - Beak.Length_Nares)
```

Which bird has the largest difference between these measurements?

Select

- Selects fields to keep (or remove)

Keep only family, species, mass:

```
fsm <- avo |>  
  select(Family1, Species1, Mass)
```

Toss range size:

```
no_rangesize <-  
  avo |> select(!Range.Size)
```

Try these out now

Select

We can also use the “:” syntax to select multiple fields

```
avo_tax <- avo |>  
  select(Species1:Order1)
```

Try this and see if you understand what it did.

How would you remove Primary.Lifestyle, Range.Size and everything in between?

Rename

Let's say we wanted to remove the 1s from the taxonomic variables:

```
avo %>%  
  rename(Species = Species1,  
         Family = Family1,  
         Order = Order1)
```

Rename

Note that you can do this during a select() as well:

```
avo_tax2 <-  
  avo |>  
  select(Species = Species1,  
         Family = Family1,  
         Order = Order1,  
         Mass)
```

Relocate

Often easier to have important variables on the left-most columns

```
avo_relocated <- avo |>  
  relocate(Order1,Family1,Species1,Mass,Wing.Length)
```

Try this and look at the `avo_relocated` dataframe.

Why are only some of the variables relocated?

Grouping

You can “group” rows of data together

- Allows you to create summary statistics for those groups
- Usually combined with another function like `summarize()`

Important functions:

`group_by()`

`ungroup()`

`summarize()`

`slice..`

Using group_by()

Let's add in family level mean mass information:

```
avo_w_family_mass <- avo |>  
  group_by(Family1) |>  
  mutate(fam_mean_mass = mean(Mass))
```

Run this and see if the results make sense.

summarize

If we ONLY wanted the data at the family level, we can use summarize:

```
family_mass <- avo |>  
  group_by(Family1) |>  
  summarise(Mean_Mass = mean(Mass))
```

Try this. How many columns and how many rows do you get back. Why these?

Slice functions

These return a subset of data

- Can be used to see just the first, the last, or a random sample
- Useful for examining a subset of data (speeds up analyses)
- Useful for subsetting biggest/smallest values

Include:

```
slice_head()  
slice_tail()  
slice_min()  
slice_max()  
slice_sample()
```

Slice functions

To get the largest bird species in each family:

```
biggest_in_fams <- avo |>  
  group_by(Family1) |>  
  slice_max(Mass, n = 1) |>  
  select(Family1, Species1, Mass)
```

How would we modify this to get the smallest bird in each family?

Before next class:

- Read “R for Data Science” chapter 5:
 - <https://r4ds.hadley.nz/data-tidy.html>
- May be a quiz